

Recon.ai

Multi-Source Payment Reconciliation Agent

Razorpay AI Buildathon 2026 — Track 04: AI Finance Controller

One-line summary: An autonomous agent that closes a multi-source payment reconciliation loop across a 50+ record synthetic batch, reporting measured match rate, throughput, accuracy, and a fully categorized, honest exception list.

1. Executive Summary

Recon.ai is an autonomous AI Finance Controller that closes the payment-to-settlement reconciliation loop. It ingests realistic synthetic records across payment/gateway settlements, bank credits, the internal ledger, and supporting events such as refunds, fees, taxes, adjustments and chargebacks. It reconciles the batch, investigates discrepancies, resolves cases only when evidence supports a valid explanation, and escalates cases where evidence is insufficient.

2. Track Fit

The project directly satisfies the AI Finance Controller requirement: close one finance-ops loop across a 50+ record synthetic batch, report match performance, and expose exceptions the agent cannot resolve. It uses the listed multi-source reconciliation direction and is designed around throughput, measured accuracy and an honest exception list.

3. Core Workflow

Load payment/gateway, bank and ledger data → normalize records → run deterministic matching → investigate ambiguous discrepancies with an AI agent → validate proposed resolutions deterministically → resolve supported cases → escalate unsupported cases → report metrics and exceptions.

4. Synthetic Data

Include a seedable generator with an independent ground-truth layer. Generate a controlled mix of exact matches, fee/tax differences, date offsets, reformatted references, partial/split settlements, many-to-one bank batches, refunds, chargebacks, adjustments, duplicates, missing records, ambiguous candidates, conflicting evidence and genuine unexplained variances. Recommended demo batch: 500 records, with larger stress-test modes if feasible.

5. Ground Truth

Ground truth defines the correct relationship and expected outcome for every synthetic case. It must be generated before source noise is introduced and must remain inaccessible to the agent during inference. Use it only after a run to calculate objective metrics.

6. Reconciliation Engine

Use a hybrid approach. Deterministic first pass: exact references/UTRs, order relationships, amount plus date windows, normalized/fuzzy references, fee/tax/net validation, duplicate checks and known refund/chargeback/adjustment explanations. Support one-to-many and many-to-one relationships, especially batched bank credits. Send only residual, ambiguous or low-confidence cases to the AI investigation layer.

7. Financial Integrity

All monetary arithmetic, fee/tax/net calculations, variance calculations, tolerances and final numerical validation must be deterministic. Prefer integer minor units or exact decimal arithmetic. The LLM reasons over evidence; it must never be the source of financial truth.

8. Investigation Agent

The agent must investigate rather than merely explain. It should identify the discrepancy, form hypotheses, select relevant evidence sources, inspect refunds/fees/chargebacks/adjustments/related settlements, calculate variance

through deterministic tools, validate the explanation and then resolve or escalate. Use structured tool calling, strict schemas, bounded steps/tool calls and timeouts.

9. Decision Policy

Use controlled states: MATCHED, RESOLVED, REVIEW, UNRESOLVED, DUPLICATE, MISSING and CONFLICT. Auto-resolve only when strong evidence and deterministic validation agree. Ambiguous or conflicting cases must go to review. If no valid evidence explains a variance, explicitly mark it UNRESOLVED and request human review.

10. Exception Center

Treat exceptions as a first-class output. For each case show category, severity, monetary impact, expected/actual values, variance, evidence checked, confidence, reason and recommended next action. Prioritize material financial risk.

11. Evaluation

Against independent ground truth, calculate match precision, match recall, resolution rate, false auto-resolution rate, exception accuracy, throughput, value reconciled and total unexplained variance. Keep match rate distinct from precision, recall and resolution rate. Never hard-code or cherry-pick metrics.

12. Dashboard & UI

Build a polished fintech operations console with Dashboard, Run Reconciliation, Transactions, Exception Center, Transaction Detail, Ask the Controller and Run History. The dashboard must dynamically show records processed, transaction value, processing time, throughput, matched/resolved/review/unresolved counts, match rate, precision, recall, resolution rate, false-resolution rate and unexplained monetary variance.

13. Transaction Detail

Show the Ledger → Gateway → Bank chain; expected versus actual amounts; variance; settlement timing; references; supporting refunds/fees/chargebacks/adjustments; matching evidence; confidence; final status; human-readable explanation; and a timestamped audit trail.

14. Ask the Controller

Provide grounded natural-language Q&A; over actual reconciliation results. Support questions such as why an order failed to reconcile, which discrepancies exceed a monetary threshold, why a case was resolved, which variances were caused by refunds, and how much unexplained variance remains. Every answer must be traceable to actual data and must not invent facts.

15. Audit Trail

Record run ID, timestamp, case ID, sources queried, evidence found, variance, rules/tools used, agent decision, confidence, validation result and resolution/escalation reason. Every transaction decision must be explainable from this record.

16. Demo Narrative

Use a deterministic 500-record dataset. Run the controller and show real progress. Display actual throughput, match rate, precision, recall, resolution rate and exceptions. Open a clean match, a discrepancy resolved through evidence such as a refund, an unresolved variance where the agent refuses to guess, then ask the controller why it was unresolved. Finish with independent evaluation metrics and exception breakdown.

17. Technical Architecture

Preferred stack: React + TypeScript, Tailwind CSS or equivalent component system, FastAPI/Python, PostgreSQL or SQLite, Python data-processing libraries, strict schema validation, an LLM with structured tool calling, and an automated evaluation harness. Keep frontend, API, data layer, generator, normalization, reconciliation, agent, evaluation, exceptions and tests modular.

18. Security & Reliability

Protect secrets with environment variables. Validate uploads and tool parameters. Treat source data as untrusted and defend against prompt injection inside transaction fields. Handle malformed data, invalid amounts/dates, duplicates, LLM timeouts, malformed model responses, database failures and empty datasets gracefully.

19. Testing

Include automated tests for generator reproducibility, schema validity, deterministic matching, fuzzy matching, dates, amounts, batching, split settlements, refunds, chargebacks, duplicates, missing records, ambiguous cases, financial calculations, agent schema handling, hallucination resistance, evaluation metrics and key UI workflows. Include adversarial false-match cases.

20. Definition of Done

The application must process at least 500 synthetic records end-to-end; use multiple financial sources; keep ground truth separate; reconcile straightforward cases; investigate realistic discrepancies; resolve evidence-backed exceptions; refuse ambiguous/unexplained cases; produce transaction-level audit trails; calculate objective metrics; expose unresolved monetary impact; provide a polished dashboard; and run reliably from a clean start with seeded demo data.

21. Product Principles

Accuracy over artificial perfection. Never force a financial match without sufficient evidence. Deterministic code for money and validation. AI for investigation and reasoning. Every decision explainable and auditable. Exceptions are valuable output, not failure. Independent ground truth. Optimize for a complete working finance-ops loop. This is not another finance chatbot; it is a finance operator.